

Reproduction Information

Probabilistic analysis of ecological, economic, and health tradeoffs of decarbonization pathways for New England, USA

Amir M. Gazar^{1,2}, Chloe Jackson³, Georgia Mavrommati³, Rich B. Howarth⁴, and Ryan S.D. Calder^{1,2,5,*}

¹Department of Population Health Sciences, Virginia Tech, Blacksburg, VA, 24061, USA

²Global Change Center, Virginia Tech, Blacksburg, VA, 24061, USA

³School for the Environment, University of Massachusetts Boston, Boston, MA 02125, USA

⁴Environmental Program, Dartmouth College, Hanover, NH, 03755, USA

⁵Department of Civil and Environmental Engineering, Duke University, Durham, NC, 27708, USA

* Corresponding author: rsdc@vt.edu

Contents

1	Conceptual Overview and Code Availability	3
1.1	Code availability	3
1.2	Conceptual Overview	3
2	Configuration and Setup	3
2.1	Hardware and Operating System	3
2.2	Software Versions and Dependencies	4
2.3	Install Time	4
2.4	Run Time	4
3	Decarbonization Pathways	4
3.1	Decarbonization Pathways Data Processing	4
4	Generation Expansion Model	5
4.1	Wind and Solar Hourly Capacity Factors	5
4.2	Small Modular Reactors (SMR)	6
4.3	Fossil Fuels: Facility Data and Emissions	6
4.4	New Fossil Fuel Facilities	7
4.5	Imports	8
4.6	Demand Analysis	9
4.7	Randomization	10
5	Dispatch Curve	11
5.1	Simulation Execution and Aggregation	11
5.2	Exporting Results	12
5.3	Validation against Deterministic ISO-NE Trajectories	12
6	Dispatch Curve Results Processing	14
6.1	Overview	14
6.2	Spatial Enrichment of Facility Results	14
6.3	Annual System Summaries	15
6.4	Key Code Snippets	15
7	Total Costs	16
7.1	CAPEX, FOM, and VOM Costs	16
7.2	Fuel Costs	18
7.3	Imports Costs	19
7.4	GHG Emissions Costs	20
7.5	Air Pollutant Emissions Costs	21
7.6	Unmet Demand Penalty Costs	22
7.7	Beyond the Fence Line Costs	24
7.8	Total Costs	25
7.9	CPI and Other Cost Variables	26
7.10	Supplemental Costs Calculation	27
8	Ecological Impacts	28
8.1	Land-use Change	28
8.2	Avian Mortality	29
8.3	Water Withdrawals	29
8.4	Population Exposure (Viewshed)	30

1 Conceptual Overview and Code Availability

1.1 Code availability

The codes for this study are available on the GitHub repository via this link.

1.2 Conceptual Overview

The framework integrates key components across three stages: (1) Input Variables, which include decarbonization pathways (e.g., new offshore wind, transmission lines to/from Quebec, and new small modular reactors) and external data sources (e.g., ISO-NE demand, EIA and EPA datasets, and IEA resources); (2) the Generation Expansion Model, which computes demand curves, clean and fossil fuel generation, imports, energy storage, and unmet demand at the Regional Transmission Organization (RTO) level; and (3) Total Costs, which are determined through cost modules covering CAPEX, FOM, VOM, emissions, fuel, imports, and unmet demand penalties to estimate total social costs.

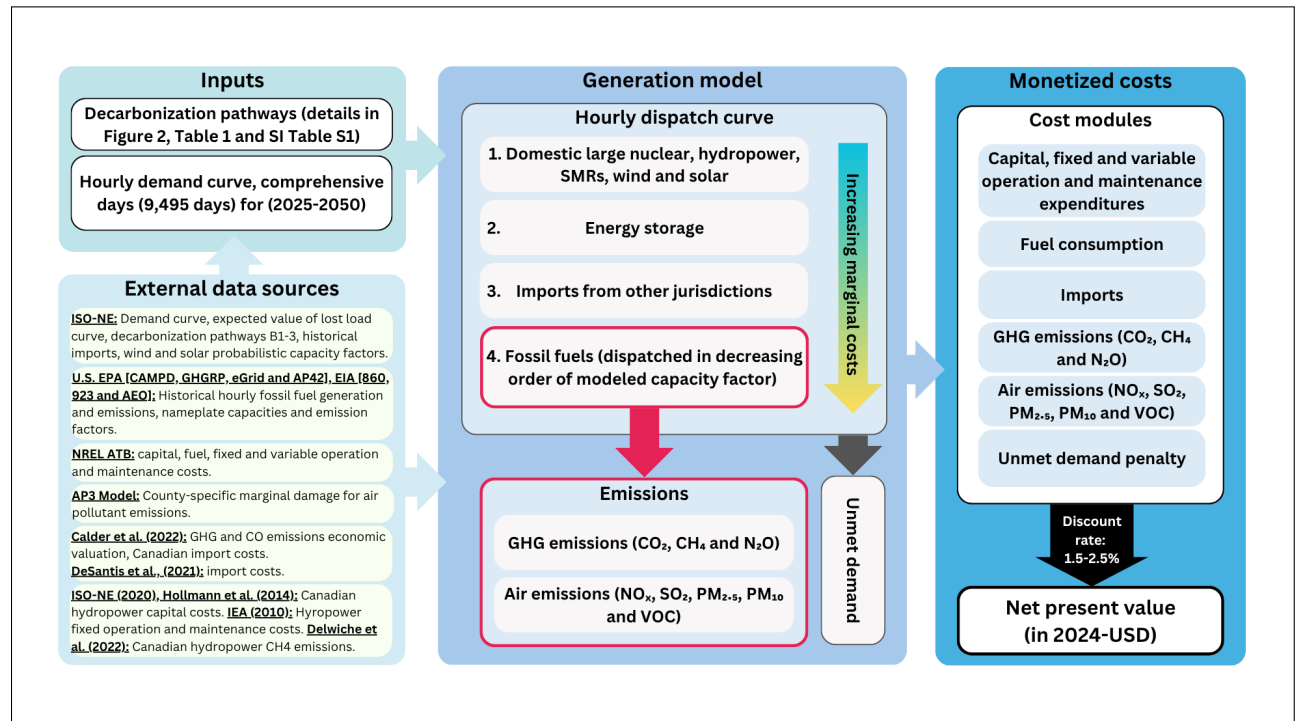


Figure 1: Conceptual overview of the modeling framework.

2 Configuration and Setup

This document presents a comprehensive guide outlining the sequential process required to execute and replicate the capacity expansion model and total cost calculation presented in our article. Additionally, we provide a conceptual overview of code functionality in Section 1.2.

2.1 Hardware and Operating System

We utilized a MacBook Pro with an Apple M1 Pro chip, featuring an 8-core CPU and 16 GB of memory. The startup disk is the Macintosh HD. The system operates on macOS 14.5 Sonoma. To run the generation expansion model script, we utilized Virginia Tech's Advanced Research Computing (ARC) resources. We used the following specifications for this source for each simulation.

```
#SBATCH --partition=normal_q
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=50
#SBATCH --cpus-per-task=1
#SBATCH --time=24:00:00
```

2.2 Software Versions and Dependencies

We used R version 4.4.2 (2024-10-31) for the x86_64-apple-darwin20 platform. Furthermore, we used the RStudio environment, version 2024.09.1 Build 394 to execute the code. Table 1 shows the packages and versions utilized in this code.

Table 1: Package Version

Package	Version
data.table	1.14.8
lubridate	1.9.3
httr	1.4.6
jsonlite	1.8.7
htmltools	0.5.5
ggplot2	3.4.2
scales	1.2.1
zoo	1.8.12
gridExtra	2.3
dplyr	1.1.2

2.3 Install Time

Installation of RStudio and R depends on the specific hardware and operating system in use. This also holds true for the packages employed within the code. However, the packages used in this script typically take a few minutes to install.

2.4 Run Time

The run time to execute the generation expansion model script in ARC at Virginia Tech is as follows:

- Full demand curve (9,495 days): 15 minutes for 1 simulation for each decarbonization pathway.

Depending on the size of the data set and the hardware configuration, this duration may vary. For all other scripts, the run time was a few minutes on our computer.

3 Decarbonization Pathways

3.1 Decarbonization Pathways Data Processing

The decarbonization pathways are analyzed using a custom R script, `Hourly_Installed_Capacity.R`, which processes data from an Excel file. The key steps are as follows:

1. **Loading Libraries**: The script utilizes the `readxl`, `data.table`, and `lubridate` libraries for reading Excel files, managing data efficiently, and handling date-time information, respectively.
2. **Reading Decarbonization Data**:
 - The Excel file, `Decarbonization_Pathways.xlsx`, contains multiple sheets, each representing a decarbonization pathway.
 - All sheets are read and combined into a unified dataset.
3. **Adding Metadata**: A new column `Pathway` is added to the data from each sheet, identifying the respective pathway.
4. **Combining Data**: The data from all sheets are combined into a single `data.table` for further analysis.
5. **Extracting Year Range**: The script identifies the range of years in the dataset (`start_year` and `end_year`) for contextualizing analysis.

Important Functions and Steps:

- Loading Excel sheet names:

Listing 1: Loading Excel Sheet Names

```
1 sheet_names <- excel_sheets(file_path)
```

- Adding pathway metadata to each sheet:

Listing 2: Adding Pathway Metadata

```
1 sheet_data[, Pathway := sheet]
```

- Combining all data:

Listing 3: Combining All Sheets

```
1 combined_data <- rbindlist(data_tables, fill = TRUE)
```

- Extracting year range:

Listing 4: Extracting Year Range

```
1 years <- unique(as.numeric(combined_data$Year))
2 start_year <- min(years, na.rm = TRUE)
3 end_year <- max(years, na.rm = TRUE)
```

This script prepares the decarbonization pathways data for subsequent analysis, ensuring all relevant information is integrated and organized.

4 Generation Expansion Model

4.1 Wind and Solar Hourly Capacity Factors

4.1.1 Overview

The `Wind.Solar.Hourly.CF.R` script processes hourly wind and solar generation data to calculate capacity factors. The script performs several preprocessing steps to standardize the data and align it with the time zones and calendar conventions used in the model.

4.1.2 Detailed Explanation

1. ****Loading Data****: Hourly generation data is read from CSV files, and the required columns are selected for analysis.
2. ****Time Zone Adjustment****: Timestamps are converted from UTC to Eastern Standard Time (EST), aligning with local conditions.
3. ****Handling Leap Years****: Leap year adjustments ensure the data is consistent, avoiding errors in models that depend on the length of the year.

4.1.3 Key Code

Listing 5: Adjusting Timestamps and Handling Leap Years

```
1 data_processing_func <- function(file, columns_to_keep) {
2   data <- readr::read_csv(file, show_col_types = FALSE)
3   data <- data %>% dplyr::select(all_of(columns_to_keep))
4   data$Date <- as.POSIXct(data$Date, format = "%m/%d/%Y %I:%M:%S %p", tz = "UTC")
5
6   # Convert UTC to EST
7   data$Date <- data$Date - hours(5)
```

```

8
9   # Add DayLabel and Hour columns
10  data <- data %>%
11    mutate(DayLabel = yday(Date), Hour = hour(Date) + 1)
12
13  # Adjust DayLabel for non-leap years
14  if (!is_leap_year(year(data$Date[365]))) {
15    data <- data %>% mutate(DayLabel = ifelse(DayLabel == 366, 365, DayLabel))
16  }
17 }

```

4.2 Small Modular Reactors (SMR)

4.2.1 Overview

The `SMR_CF.R` script models the operational characteristics of small modular reactors. These include:

- Capacity factor (90%)
- Ramp rate and minimum power output
- Core fuel characteristics, including enrichment and burnup

4.2.2 Key Code

Listing 6: Defining SMR Specifications

```

1  data <- data.frame(
2    Label = "SMR300",
3    Category = "SMR",
4    Nameplate.Capacity.MW = 300,
5    CF = 0.9,
6    Ramp = 1,
7    Minimum.Power.Output.MW = 60,
8    Enrichment = "4.95%",
9    Fuel = "UO2",
10   Reactor.Lifetime = 60
11 )

```

4.3 Fossil Fuels: Facility Data and Emissions

4.3.1 Fossil Fuel Facility Data

The `Fossil.Fuels.Facilities.Data.R` script processes historical fossil fuel facility data for New England. This data is critical for understanding the existing capabilities and limitations of the regional fossil fuel infrastructure.

Detailed Explanation:

1. **Loading Data**: Facility data for all U.S. states is loaded. The dataset includes information on facility location, capacity, operating status, and ramp rates.
2. **Filtering for Active Facilities**: Facilities are filtered to include only those located in New England and those that are operational (i.e., not retired).
3. **Ramp Rate Conversion**: Ramping times (e.g., "10M", "1H") are converted into numeric values representing hours. This standardization ensures compatibility with the model.

Key Code Excerpts:

Listing 7: Ramp Rate Conversion Function

```

1  convert_ramp_to_numeric <- function(ramp) {
2    if (is.na(ramp)) {
3      return(24) # Default to 24 hours for missing values

```

```

4   } else if (ramp == "10M") {
5     return(0.1666667) # 10 minutes
6   } else if (ramp == "1H") {
7     return(1) # 1 hour
8   } else if (ramp == "12H") {
9     return(12) # 12 hours
10  } else if (ramp == "OVER") {
11    return(24) # Default for 'OVER'
12  } else {
13    return(24)
14  }
15 }

```

Listing 8: Filtering Facilities for New England

```

1 # Filter data for active facilities in New England
2 new_england_states <- c("CT", "ME", "MA", "NH", "RI", "VT")
3 facilities_data_NE <- facilities_data[State %in% new_england_states]
4 facilities_data_NE <- facilities_data_NE[!grepl("retired", Operating_Status, ignore.case = ...
  TRUE)]

```

4.3.2 Fossil Fuel Generation Emissions

The `Fossil.Fuels.Gen.Emissions.R` script compiles hourly emissions data for fossil fuel facilities in New England. Emissions data is crucial for estimating environmental costs and compliance with emission regulations.

Detailed Explanation:

1. **Reading Emissions Data**: Hourly emissions data is read for each New England state.
2. **Combining Emissions Data**: Data from individual states is combined into a single dataset.
3. **Cross-Referencing with Facility Data**: Emissions data is filtered to include only active facilities, ensuring consistency with the facility dataset.

Key Code Excerpts:

Listing 9: Combining Emissions Data

```

1 combined_emissions_data <- rbindlist(all_emissions_data, fill = TRUE)
2
3 # Filter emissions data to match active facilities
4 active_facilities <- facilities_data_NE$Facility_Unit.ID
5 combined_emissions_data <- combined_emissions_data[Facility_Unit.ID %in% active_facilities]

```

4.4 New Fossil Fuel Facilities

4.4.1 Overview

The `Fossil.Fuels.New.R` script models the addition of new fossil fuel facilities to meet future energy demands. These facilities are phased in starting from 2033, with three units per facility and one facility added annually.

Detailed Explanation:

1. **Defining Facility Specifications**: Each facility is given a unique ID, and the year it comes online is specified.
2. **Expanding Facility Data**: Each facility is expanded into three units to represent its operational characteristics accurately.

Key Code Excerpts:

Listing 10: Defining New Fossil Fuel Facilities

```
1 # Define 18 facilities starting in 2033
2 new_facilities <- data.table(
3   Facility_ID = paste0("NGCC", 3000:(3000 + num_facilities - 1)),
4   Year_Online = start_year:(start_year + num_facilities - 1)
5 )
6
7 # Expand each facility into 3 units
8 new_facility_units <- new_facilities[, .(
9   Unit_ID = 1:3,
10  Facility_ID,
11  Year_Online
12 )]
```

4.5 Imports

4.5.1 Overview

The `imports.R` script processes electricity imports data to calculate statistics and capacity factors for modeling electricity imports into the region. Key components of this script include:

- Loading daily imports data for the years 2011 to 2023 from an Excel file.
- Extracting the maximum capacity factors for transmission lines based on NREL ATB standards (set to 95%).
- Computing percentiles for electricity imports over the given time period.

4.5.2 Detailed Explanation

1. ****Loading Libraries****: The script uses a range of libraries, including `dplyr`, `readxl`, `lubridate`, `data.table`, and `tidyverse` for data manipulation, reading Excel files, and date handling.
2. ****Defining Time Range****: The imports data is analyzed for the period between January 1, 2011, and December 31, 2023.
3. ****Loading Data****:
 - The script identifies and reads sheets from the Excel file corresponding to years between 2011 and 2023.
 - Data from each sheet is combined into a single dataset, appending a `Year` column for clarity.
4. ****Percentile Calculations****: The `calculate_percentiles` function calculates the percentiles (from 1% to 99%) of a specified column in the dataset. This is useful for understanding the distribution of electricity imports.

4.5.3 Key Code

Defining Time Range and Max Capacity Factor The time range for the analysis and the maximum capacity factor for transmission lines (set to 95%) are defined as follows:

Listing 11: Defining Time Range and Max Capacity Factor

```
1 start_date <- as.Date("2011-01-01")
2 end_date <- as.Date("2023-12-31")
3 max_CF_transmission_lines <- 0.95 # According to NREL ATB
```


Loading Imports Data The imports data is loaded from an Excel file, extracting only the sheets corresponding to the years of interest (2011–2023):

Listing 12: Loading Electricity Imports Data

```
1 file <- "daily_capacity_status.xlsx"
2 imports_data <- data.frame()
3 years <- as.character(2011:2023)
4 sheet_names <- excel_sheets(file)
5 sheet_names_of_interest <- sheet_names[sapply(sheet_names, function(name) name %in% years)]
6
7 for (sheet_name in sheet_names_of_interest) {
8   sheet_data <- read_excel(file, sheet = sheet_name)
9   sheet_data$Year <- sheet_name
10  imports_data <- rbind(imports_data, sheet_data)
11 }
```

Calculating Percentiles The following function calculates percentiles for a specified column of the imports dataset:

Listing 13: Percentile Calculation Function

```
1 calculate_percentiles <- function(data, column) {
2   quantiles <- quantile(data[[column]], probs = seq(0.01, 0.99, by = 0.01), na.rm = TRUE)
3   tibble(Percentile = seq(1, 99), Value = quantiles)
4 }
```

This function is applied to the imports data to understand the variability and trends in electricity imports.

4.5.4 Insights and Results

The `imports.R` script prepares the data for modeling electricity imports, providing:

- A unified dataset of daily electricity imports from 2011 to 2023.
- Insights into the statistical distribution of imports through percentile calculations.
- Capacity factor assumptions for transmission infrastructure.

4.6 Demand Analysis

4.6.1 Overview

The script `demand.R` processes hourly demand data to ensure consistency and accuracy, especially for leap years. Its main functionalities include:

- Estimating missing February 29th demand values in leap years.
- Cleaning and standardizing hourly demand data for the energy model.

4.6.2 Leap Year Adjustment

The script checks for leap years and fills missing February 29th demand data by averaging February 28th and March 1st values for each hour. The following code performs this operation:

Listing 14: Filling Missing Leap Year Data

```
1 fill_leap_year_data <- function(data) {
2   for (year in unique(data$Year)) {
3     if (year %% 4 == 0 && (year %% 100 != 0 || year %% 400 == 0)) { # Check if leap year
4       for (hour in 0:23) {
5         feb_28_data <- data[Year == year & Month == 2 & Day == 28 & Hour == hour, Demand]
6         mar_1_data <- data[Year == year & Month == 3 & Day == 1 & Hour == hour, Demand]
7
8         if (length(feb_28_data) == 1 && length(mar_1_data) == 1) {
```

```

9         leap_day_demand <- mean(c(feb_28_data, mar_1_data), na.rm = TRUE)
10        data <- rbind(data, data.table(
11            Year = year,
12            Month = 2,
13            Day = 29,
14            Hour = hour,
15            Demand = leap_day_demand
16        ))
17    }
18 }
19 }
20 }
21 return(data)
22 }

```

4.7 Randomization

4.7.1 Overview

The `randomization.R` script generates pseudo-random sequences for use in probabilistic modeling. These sequences are designed to represent variability in key parameters or inputs to the model. The script performs the following tasks:

- Defines parameters for the randomization process, including the number of sequences and the length of each sequence.
- Uses a custom function to generate pseudo-random sequences.
- Saves the generated sequences to a CSV file for integration into the modeling framework.

4.7.2 Detailed Explanation

1. **Loading Libraries**: The `randtoolbox` library is loaded, which provides tools for generating random and quasi-random numbers.
2. **Defining Parameters**:
 - `num_sequences`: The total number of sequences to generate (1,000,000).
 - `num_numbers`: The length of each sequence (250 numbers per sequence).

These parameters define the scope and granularity of the randomization process.

3. **Generating Pseudo-Random Sequences**: A custom function, `generate_pseudo_random_sequences`, is defined to generate sequences of pseudo-random numbers. This function uses the `sample` function to create a sequence of integers between 1 and 99.
4. **Saving Random Sequences**: The generated sequences are saved to a CSV file for later use in the generation expansion model.

4.7.3 Key Code

Defining the Randomization Function The following function generates pseudo-random sequences:

Listing 15: Pseudo-Random Sequence Generator

```

1 generate_pseudo_random_sequences <- function(num_sequences, num_numbers) {
2   random_sequences <- replicate(num_sequences, sample(1:99, num_numbers, replace = TRUE), ...
3     simplify = FALSE)
4   return(random_sequences)
5 }

```

Generating and Transposing Random Sequences The generated sequences are combined into a matrix and transposed for storage:

Listing 16: Generating and Transposing Random Sequences

```
1 # Parameters
2 num_sequences <- 1e6
3 num_numbers <- 250
4
5 # Generate Pseudo-random sequences
6 Percentile_sequences_random <- generate_pseudo_random_sequences(num_sequences, num_numbers)
7
8 # Convert the list to a matrix and transpose it
9 Percentile_sequences_matrix <- t(do.call(rbind, Percentile_sequences_random))
```

Saving to a CSV File The pseudo-random sequences are saved to a CSV file for subsequent use:

Listing 17: Saving Random Sequences to CSV

```
1 file_path_random_csv <- "Random.Sequence.csv"
2 write.csv(Percentile_sequences_matrix, file_path_random_csv, row.names = FALSE)
```

4.7.4 Insights and Applications

The randomization process enables:

- Creation of large-scale probabilistic datasets for use in energy modeling.
- Representation of uncertainty and variability in model parameters.
- Efficient storage of sequences for repeated use across simulations.

The generated pseudo-random sequences are integrated into the generation expansion model, adding flexibility and robustness to the analysis.

5 Dispatch Curve

Once the core dispatch-curve routines have produced hourly and per-unit outputs, we run end-to-end simulations across all decarbonization pathways, aggregate the results, export them, and generate basic diagnostic plots.

5.1 Simulation Execution and Aggregation

We iterate over each simulation index and each pathway, calling in turn:

1. `dispatch_curve(sim, pathway)` to compute the clean-plus-storage dispatch curve,
2. `dispatch_curve_adjustments()` to sample and adjust fossil-fuel outputs under ramp constraints,
3. `dispatch_curve_calibrations()` to recombine fossil outputs and re-run battery/import adjustments.

Each function returns a `data.table` tagged with `Simulation` and `Pathway`. We then flatten the nested lists into two master tables: one for the final hourly dispatch results and one for the facility-level fossil fuel outputs.

Listing 18: Running simulations and combining results

```
1 # 1. Define simulations & pathways
2 pathways <- unique(Hourly_Installed.Capacity$Pathway)
3 n_simulations <- 1
4
5 # 2. Execute each sim and pathway
6 simulation_results <- lapply(1:n_simulations, function(sim) {
7   lapply(pathways, function(pathway) {
```

```

8   dc.results    <- dispatch.curve(sim, pathway)
9   ff.adjusted   <- dispatch.curve.adjustments(dc.results)
10  final.hourly  <- dispatch.curve.calibrations(dc.results, ff.adjusted)
11  # Tag with sim & pathway
12  dc.results$Simulation <- ff.adjusted$Simulation <- final.hourly$Simulation <- sim
13  dc.results$Pathway   <- ff.adjusted$Pathway   <- final.hourly$Pathway   <- pathway
14  list(final.hourly = final.hourly, fossil.fuels = ff.adjusted)
15 })
16 })
17
18 # 3. Flatten into two data.tables
19 combined.final.hourly.results <- do.call(rbind, lapply(simulation.results, function(s) {
20   do.call(rbind, lapply(s, `[`, "final.hourly"))
21 }))
22 combined.fossil.fuels.results <- do.call(rbind, lapply(simulation.results, function(s) {
23   do.call(rbind, lapply(s, `[`, "fossil.fuels"))
24 })))

```

5.2 Exporting Results

After aggregation, we write the two tables to CSV for downstream analysis or visualization.

Listing 19: Writing aggregated results to CSV

```

1 # Save combined hourly dispatch results
2 write.csv(combined.final.hourly.results,
3           "/path/to/Hourly.Results.NE.csv",
4           row.names = FALSE)
5
6 # Save combined facility level fossil outputs
7 write.csv(combined.fossil.fuels.results,
8           "/path/to/Facility.Level.Results.csv",
9           row.names = FALSE)

```

5.3 Validation against Deterministic ISO-NE Trajectories

To check that the dispatch-curve implementation reproduces the intended aggregate dynamics, we validate the model against the deterministic ISO-NE trajectories from the Massachusetts 2050 Decarbonization Roadmap workbook. Specifically, for a given decarbonization pathway and scenario (for example B1 under High Electrification), we compare annual total load and renewable generation from the model against the Roadmap benchmarks for years 2025 to 2050 in five-year steps.

We use the yearly summary output from the dispatch model (`Yearly.Results.csv`) and an external Excel workbook containing the ISO-NE benchmark trajectories. Paths and scenario metadata are set at the top of the validation script:

Listing 20: Validation settings and input files (snippet)

```

1 target.scenario <- "High Electrification"
2 target.pathway  <- "B1"
3 years.to.validate <- seq(2025, 2050, by = 5)
4
5 proj_dir <- "/path/to/project/root"
6 model.file <- file.path(proj_dir, "2 Generation Expansion Model",
7                          "5 Dispatch Curve", "4 Final Results",
8                          "1 Comprehensive Days Summary Results",
9                          "Yearly.Results.csv")
10 excel.file <- file.path(proj_dir, "4 External Data",
11                          "Massachusetts 2050 Decarbonization Roadmap Study",
12                          "Massachusetts Workbook of Energy Modeling Results 2024.xlsx")

```

Model-side annual metrics. From the yearly model output, we construct total annual load and total annual renewables for each simulation and year. Total load is defined to be consistent with the Roadmap accounting (all generation including imports, with no curtailment subtraction):

Listing 21: Annual model aggregates (snippet)

```

1 model_data <- fread(model_file) [
2   Pathway == target_pathway & Year %in% years_to_validate
3 ]
4
5 model_sims <- model_data[, .(
6   Renewables = Solar.TWh + Onshore.TWh + Offshore.TWh +
7     Hydro.TWh + Biomass.TWh,
8   Total_Load = (Old.Fossil.Fuels.adj.TWh + New.Fossil.Fuel.TWh) +
9     Nuclear.TWh +
10     (Solar.TWh + Onshore.TWh + Offshore.TWh +
11     Hydro.TWh + Biomass.TWh) +
12     Calibrated.Total_import_net.TWh
13 ), by = .(Simulation, Year)]
14
15 model_stats <- model_sims[, .(
16   Model_Renewables = median(Renewables),
17   Model_TotalLoad = median(Total_Load)
18 ), by = Year]

```

Benchmark trajectories from the Roadmap workbook. The benchmark series are extracted from the Roadmap Excel workbook (sheet "10. Electricity Generation"). For each validation year we locate the appropriate scenario-year column, sum renewable rows (solar, wind, hydro), and reconstruct total generation by adding nuclear, imports via transmission, and fossil generation:

Listing 22: Benchmark extraction (conceptual snippet)

```

1 raw_gen <- as.data.table(
2   read_excel(excel_file, sheet = "10. Electricity Generation",
3     col_names = FALSE)
4 )
5
6 get_benchmark <- function(yr) {
7   # identify column for target_scenario and yr
8   # read clean generation summary block
9   # sum renewables (solar, wind, hydro)
10  # read fossil rows from top section (gas, coal, oil)
11  # add nuclear and transmission imports
12  data.table(
13    Year = yr,
14    Bench_Renewables = renew_val,
15    Bench_TotalLoad = total_val
16  )
17 }
18
19 bench_results <- rbindlist(
20   lapply(years_to_validate, get_benchmark)
21 )

```

Goodness of fit and plots. We then merge the model median series with the benchmark series and compute simple coefficient of determination (R^2) statistics for both total load and renewables:

Listing 23: R-squared calculation (snippet)

```

1 validation_df <- merge(model_stats, bench_results, by = "Year")
2
3 calc_r2 <- function(obs, pred) {
4   ss_res <- sum((obs - pred)^2)
5   ss_tot <- sum((obs - mean(obs))^2)
6   1 - ss_res / ss_tot
7 }
8
9 r2_load <- calc_r2(validation_df$Bench_TotalLoad,
10   validation_df$Model_TotalLoad)
11 r2_renew <- calc_r2(validation_df$Bench_Renewables,

```

```
12 validation_df$Model_Renewables)
```

Finally, we generate a diagnostic figure with two panels: one for total annual load and one for renewable generation. Each panel shows the median model trajectory against the Roadmap benchmark, with the corresponding R^2 value annotated:

Listing 24: Diagnostic plotting (snippet)

```
1 plot_data <- rbind(
2   melt(validation_df[, .(
3     Year, Model = Model.TotalLoad,
4     Benchmark = Bench.TotalLoad)],
5   id.vars = "Year")[, Metric := "Total Annual Load"],
6   melt(validation_df[, .(
7     Year, Model = Model.Renewables,
8     Benchmark = Bench.Renewables)],
9   id.vars = "Year")[, Metric := "Renewable Generation"]
10 )
11
12 ggplot(plot_data, aes(Year, value, color = variable)) +
13   geom_line() + geom_point() +
14   facet_wrap(~Metric, scales = "free-y") +
15   theme_minimal()
```

These validation statistics and plots provide a compact check that the dispatch-curve module reproduces the intended deterministic Roadmap trajectories for both total energy demand and renewable generation along t

6 Dispatch Curve Results Processing

6.1 Overview

After generating hourly dispatch and fossil-fuel outputs, we perform two post-processing steps:

1. **Spatial enrichment:** assign each facility to its county (and state) via a spatial join against U.S. Census boundaries, fixing Connecticut planning regions manually where needed.
2. **Annual system summaries:** compute key metrics—renewable penetration, battery discharge, short-ages, and CO₂ emissions—by year, simulation, and pathway, then combine across all scenarios.

6.2 Spatial Enrichment of Facility Results

1. Read the facility-level summary CSV into a `data.table`, drop any phantom columns.
2. Filter out rows missing latitude/longitude, convert to an `sf` object (WGS84).
3. Load county geometries (via `tigris::counties` plus a New England GeoJSON), ensure both use CRS 4326.
4. Spatially join facilities to counties (`st_within`), then:
 - For CT, map planning-region names to county names and GEOIDs.
 - Manually correct any unmatched units (e.g. facility IDs “10823_S42”, “10823_S43” to Suffolk, MA).
5. Save the enriched table (with `County_State`, `GEOID`, etc.) to CSV.

6.3 Annual System Summaries

1. For each hourly-results CSV, compute:

$$\text{RenewablePenetration} = \frac{\text{Solar_MWh} + \text{Onshore_MWh} + \text{Offshore_MWh}}{\text{Clean_MWh} + \text{Imports_MWh} + \text{Fossil_MWh} + \text{NewFossil_MWh}},$$

$$\text{BatteryDischarges_GWh} = \sum (\text{Calibrated_Battery_discharge}) / 10^3,$$

$$\text{Shortages_TWh} = \sum (\text{Calibrated_Shortage_MWh}) / 10^6,$$

$$\text{CO2_tons} = \sum (\text{CO2_tons}).$$

2. Summarize these by Year, Simulation, and Pathway.
3. Row-bind across all files and write Battery_Data.csv.

6.4 Key Code Snippets

Listing 25: Spatial join and county assignment

```

1  #--- Load and clean facility results
2  Yearly_Fac <- fread(file_path)[, V1 := NULL]
3  fac_clean <- Yearly_Fac[!is.na(latitude) & !is.na(longitude)]
4  fac_sf <- st_as_sf(fac_clean, coords=c("longitude", "latitude"), crs=4326)
5
6  #--- Load counties, spatial join
7  counties <- counties(cb=TRUE, class="sf") %>% st_transform(4326)
8  fac_sf <- st_join(fac_sf, counties, join=st_within)
9
10 #--- Fix CT planning regions, county names and GEOIDs
11 fac_dt <- as.data.table(fac_sf)
12 fac_dt[STUSPS=="CT", NAME := planning_to_county[NAME]]
13 fac_dt[STUSPS=="CT", GEOID := county_to_geoid[NAME]]
14 #--- Manual fixes for known mismatches...
15 fwrite(fac_dt, file.path(output_path, "Yearly-Facility-Level-Results-County-added.in.csv"))

```

Listing 26: Annual summary of renewables, storage, shortages, CO2

```

1  #--- Read all hourly outputs, compute per-file summaries
2  files <- list.files(folder_path, "^Hourly-Results-.*\\.csv$", full.names=TRUE)
3  summary_list <- list()
4  for(f in files) {
5    dt <- fread(f)
6    dt[, Renewable.Penetration :=
7      (Solar_MWh+Onshore_MWh+Offshore_MWh) /
8      (Calibrated.Total.import.net_MWh+Old.Fossil.Fuels_adj_MWh+Clean_MWh+New.Fossil.Fuel_MWh)]
9    summary_list[[f]] <- dt[, .(
10      RP_mean = mean(Renewable.Penetration, na.rm=TRUE),
11      RP_max = max(Renewable.Penetration, na.rm=TRUE),
12      RP_min = min(Renewable.Penetration, na.rm=TRUE),
13      Battery_Discharges_GWh = sum(Calibrated.Battery_discharge, na.rm=TRUE) / 1e3,
14      Shortages_TWh = sum(Calibrated.Shortage_MWh, na.rm=TRUE) / 1e6,
15      CO2_tons = sum(CO2_tons, na.rm=TRUE)
16    ), by=. (Year, Simulation, Pathway)]
17  }
18  combined_summary <- rbindlist(summary_list)
19  write_csv(combined_summary,
20    file.path(output_path, "Battery_Data.csv"),
21    row.names=FALSE)

```

7 Total Costs

7.1 CAPEX, FOM, and VOM Costs

7.1.1 CAPEX and FOM for Fossil Energy

The `1_CAPEX_FOM_ATB_Fossil.R` script calculates capital expenditures (CAPEX) and fixed operation and maintenance costs (FOM) for fossil fuel-based facilities. It performs the following tasks:

- Loads ATB (Annual Technology Baseline) cost data for fossil fuels.
- Calculates net present value (NPV) for CAPEX and FOM using a discount rate of 2%.
- Extracts data for various fossil fuel types, such as coal, gas, oil, and wood.

Key Code:

Listing 27: Calculating NPV for Fossil CAPEX and FOM

```
1 calculate_npv <- function(dt, rate, base_year) {
2   npv <- sum(dt[,2] / (1 + rate)^(dt[,1] - base_year))
3   return(npv)
4 }
5
6 discount_rate <- 0.02
7 base_year <- 2023
8
9 Coal_NPC <- Fossil_Fuels_NPC[grepl("Coal", Primary.Fuel.Type, ignore.case = TRUE)]
10 Gas_CC_NPC <- Fossil_Fuels_NPC[grepl("Gas", Primary.Fuel.Type, ignore.case = TRUE) &
11   grepl("Combined", Unit.Type, ignore.case = TRUE)]
```

7.1.2 CAPEX and FOM for Non-Fossil Energy

The `2_CAPEX_FOM_ATB_Non_Fossil.R` script focuses on renewable and nuclear energy sources. It:

- Loads ATB data for non-fossil technologies, including solar, onshore wind, offshore wind, nuclear, and hydro.
- Calculates CAPEX and FOM for each technology across different scenarios (Advanced, Moderate, Conservative).

Key Code:

Listing 28: Loading and Setting Non-Fossil Data

```
1 ATBe <- fread("ATBe.csv")
2 path <- "Installed Capacity and CF Non Fossil"
3 files <- list.files(path = path, pattern = "\\*.csv$", full.names = TRUE)
4
5 datasets <- list()
6 for (file in files) {
7   dataset_name <- gsub(".*|\\*.csv$", "", file)
8   datasets[[dataset_name]] <- fread(file)
9 }
10
11 list2env(datasets, envir = .GlobalEnv) # Release datasets into the environment
12 setDT(Solar_NPC)
13 setDT(Nuclear_hydro_bio_NPC)
```

7.1.3 CAPEX and FOM for Imports

The `3_CAPEX_FOM_Imports.R` script calculates CAPEX and FOM costs for import energy capacity. Key operations include:

- Loading capacity data for imports across three scenarios (S1, S2, S3).
- Calculating new capacity built each year and the associated costs.

Key Code:

Listing 29: Loading and Calculating Import Capacity

```
1 Imports_S1 <- read_excel("Import Capacity.xlsx", sheet = "S1")
2 Imports_S2 <- read_excel("Import Capacity.xlsx", sheet = "S2")
3 Imports_S3 <- read_excel("Import Capacity.xlsx", sheet = "S3")
4
5 Imports_Capacity <- rbindlist(list(Imports_S1, Imports_S2, Imports_S3), use.names = TRUE, ...
  fill = TRUE)
6 Imports_Capacity[, new_capacity := QC - shift(QC, 1, type = "lag"), by = Scenario]
```

7.1.4 VOM for Fossil Energy

The 4_VOM_ATB_Fossil.R script calculates variable operation and maintenance (VOM) costs for fossil fuels:

- Merges fossil fuel generation data with facility-specific data.
- Calculates yearly VOM costs for different fossil fuel technologies, accounting for scenarios in the ATB data.

Key Code:

Listing 30: Calculating VOM Costs for Fossil Fuels

```
1 Yearly_Facility_Level_Results <- selected_cols[Yearly_Facility_Level_Results, on = ...
  "Facility_Unit.ID"]
2 Yearly_Facility_Level_Results[, VOM_Cost := Generation * VOM_Rate, by = Unit.Type]
```

7.1.5 VOM for Non-Fossil Energy

The 5_VOM_ATB_Non_Fossil.R script calculates VOM costs for renewable and nuclear energy. It:

- Processes VOM costs for solar, wind, hydro, and nuclear technologies.
- Filters data based on ATB scenarios and technology-specific parameters.

Key Code:

Listing 31: Processing VOM Costs for Non-Fossil Technologies

```
1 process_non_fossil <- function(technology, techdetail, dataset, column_name, simulation, ...
  scenario) {
2   dataset <- dataset[Technology == technology & TechDetail == techdetail]
3   dataset[, VOM_Cost := Generation * VOM_Rate]
4   return(dataset)
5 }
```

7.1.6 Insights and Applications

These scripts collectively provide comprehensive cost assessments for CAPEX, FOM, and VOM across fossil, non-fossil, and imported energy sources. The outputs are critical for evaluating the economic feasibility of different decarbonization pathways under various scenarios.

7.2 Fuel Costs

7.2.1 Fuel Costs for Fossil Energy

The `7_Fuel_ATB_Fossil.R` script calculates fuel costs for fossil fuel energy sources using the NREL ATB (Annual Technology Baseline) dataset. Key operations include:

- Loading CPI (Consumer Price Index) data via the `fredr` API to adjust costs to 2024 dollars.
- Loading ATB fuel cost data for fossil fuels, including coal, natural gas, and oil.
- Calculating Net Present Value (NPV) of fuel costs using a 2% discount rate.

Key Code:

Listing 32: Loading and Adjusting Fuel Costs for Fossil Fuels

```

1 # Set discount rate and base year
2 discount_rate <- 0.02
3 base_year <- 2023
4
5 # Load CPI data to calculate conversion rate
6 cpi_data <- fredr(series_id = "CPIAUCSL", observation_start = as.Date("2000-01-01"), ...
  observation_end = as.Date("2024-01-01"))
7 cpi_2021 <- filter(cpi_data, year(date) == 2021) %>% summarise(YearlyAvg = mean(value))
8 cpi_2024 <- filter(cpi_data, year(date) == 2024) %>% summarise(YearlyAvg = mean(value))
9 conversion_rate <- cpi_2024$YearlyAvg / cpi_2021$YearlyAvg
10
11 # Load ATB fuel cost data
12 file_path <- "ATB.2021.Fuel.Costs.csv"
13 ATB_Fuel_Costs <- fread(file_path)
14 ATB_Fuel_Costs[, Adjusted_Cost := Original_Cost * conversion_rate]
```

This script adjusts fossil fuel costs to 2024 dollars and calculates NPVs to evaluate long-term fuel expenditure.

7.2.2 Fuel Costs for Non-Fossil Energy

The `8_Fuel_ATB_Non_Fossil.R` script processes fuel costs for non-fossil energy sources such as nuclear, hydro, and biomass. Its primary functions include:

- Loading ATB cost data for non-fossil fuel types.
- Filtering data based on technology, scenarios, and simulation parameters.
- Calculating VOM (Variable Operation and Maintenance) costs where applicable.

Key Code:

Listing 33: Processing Non-Fossil Fuel Costs

```

1 process_non_fossil <- function(technology, techdetail, dataset, column.name, simulation, ...
  scenario) {
2   dataset <- dataset[Technology == technology & TechDetail == techdetail]
3   dataset[, Adjusted_Cost := Fuel_Cost * VOM_Rate]
4   return(dataset)
5 }
6
7 # Example: Processing nuclear fuel costs
8 process_non_fossil("Nuclear", "Advanced", ATBe, "Fuel_Cost", "Simulation1", "Scenario1")
```

This script enables scenario-based analysis of non-fossil fuel costs under varying economic and technological assumptions.

7.2.3 Insights and Applications

These scripts provide a detailed analysis of fuel costs for fossil and non-fossil energy sources:

- Fossil fuel costs are adjusted to 2024 dollars using CPI data and evaluated for long-term feasibility.
- Non-fossil fuel costs are integrated into scenario-based models to assess their economic competitiveness.

The outputs support informed decision-making in decarbonization pathway modeling.

7.3 Imports Costs

7.3.1 Overview

The `6_Imports.R` script calculates costs related to electricity imports. Key functionalities include:

- Adjusting costs to 2024 dollars using Consumer Price Index (CPI) data retrieved from the **fredr** API.
- Calculating Net Present Value (NPV) of imports costs using a discount rate of 2%.
- Processing capacity data across multiple scenarios to estimate annual costs.

7.3.2 Detailed Explanation

1. **Loading CPI Data**:

- The script retrieves CPI data for 2021, 2022, and 2024 to adjust historical costs to 2024 values.
- Conversion rates are calculated based on the ratio of CPI values.

2. **Loading Capacity Data**:

- Import capacity data is loaded from an Excel file, with separate sheets representing different scenarios (e.g., S1, S2, S3).
- New capacity additions are calculated for each year within each scenario.

3. **NPV Calculation**: The script calculates the NPV of costs for each scenario, providing a long-term cost assessment.

7.3.3 Key Code

CPI Adjustments:

Listing 34: Loading and Adjusting CPI Data

```
1 # Set API key and retrieve CPI data
2 fredr.set_key("63522eae4ec927d6f1d9d86bf7826cc8")
3 cpi_data <- fredr(series_id = "CPIAUCSL", observation_start = as.Date("2000-01-01"), ...
  observation_end = as.Date("2024-01-01"))
4
5 # Calculate conversion rates
6 cpi_2021 <- filter(cpi_data, year(date) == 2021) %>% summarise(YearlyAvg = mean(value))
7 cpi_2024 <- filter(cpi_data, year(date) == 2024) %>% summarise(YearlyAvg = mean(value))
8 conversion_rate_2021 <- cpi_2024$YearlyAvg / cpi_2021$YearlyAvg
```

Loading Capacity Data:

Listing 35: Loading Import Capacity Data

```
1 path <- "Imports.Capacity.xlsx"
2
3 # Load data for different scenarios
4 Imports_S1 <- read_excel(path, sheet = "S1")
5 Imports_S2 <- read_excel(path, sheet = "S2")
6 Imports_S3 <- read_excel(path, sheet = "S3")
7
```

```

8 # Combine and calculate new capacity
9 Imports.Capacity <- rbindlist(list(Imports_S1, Imports_S2, Imports_S3), use.names = TRUE, ...
  fill = TRUE)
10 Imports.Capacity[, new_capacity := QC - shift(QC, 1, type = "lag"), by = Scenario]

```

NPV Calculation:

Listing 36: Calculating NPV for Import Costs

```

1 calculate_npv <- function(dt, rate, base_year, col) {
2   npv <- sum(dt[[col]] / (1 + rate)^(dt[['Year']] - base_year))
3   return(npv)
4 }
5
6 discount_rate <- 0.02
7 base_year <- 2023
8
9 # Example NPV calculation
10 npv_scenario1 <- calculate_npv(Imports.Capacity[Scenario == "S1"], discount_rate, ...
  base_year, "new_capacity")

```

7.3.4 Insights and Applications

This script provides a detailed assessment of import costs:

- Adjusting for inflation ensures that all costs are consistent with 2024 dollars.
- Scenario-based analysis enables a comparison of costs across different import strategies.
- NPV calculations provide a long-term perspective on import-related expenditures.

7.4 GHG Emissions Costs

7.4.1 Overview

The `9_GHG_Emissions.R` script calculates costs associated with GHG emissions. Key functionalities include:

- Loading annual GHG emissions data in metric tons CO₂-equivalent (IPCC AR4 GWP).
- Interpolating social costs for CO₂, CH₄, and N₂O from 2025 to 2050.
- Adjusting those costs to 2024 USD via CPI data from the `fredr` API.
- Calculating the Net Present Value (NPV) of total GHG costs at a 2.5 % discount rate.

7.4.2 Detailed Explanation

1. **Loading Emissions Data** Read facility-level and system-level CSVs of annual GHG emissions (all in metric tons CO₂-eq).
2. **CPI Adjustment** Fetch CPI series (2000–2024) via `fredr`, compute the 2020→2024 conversion factor, and scale all future SCC values.
3. **Cost Interpolation** Define SCC endpoints in 2025 and 2050 for CO₂, CH₄, N₂O; linearly interpolate for each year 2025–2050.
4. **NPV Calculation** For each simulation and pathway, sum annual

$$\text{Cost}_t = \text{CO}_{2,t} \times c_{\text{CO}_2,t} + \text{CH}_{4,t} \times c_{\text{CH}_4,t} + \text{N}_2\text{O}_t \times c_{\text{N}_2\text{O},t}$$

$$\text{then discount from 2025–2050 at 2.5 \%: NPV} = \sum_{t=2025}^{2050} \frac{\text{Cost}_t}{(1+0.025)^{t-2024}}.$$

7.4.3 Key Code

CPI cost curve:

```

1 discount_rate <- 0.025
2 base_year    <- 2024
3
4 fredr_set_key("63522eae4ec927d6f1d9d86bf7826cc8")
5 cpi <- fredr("CPIAUCSL", as.Date("2000-01-01"), as.Date("2024-01-01"))
6 f20 <- mean(filter(cpi, year(date)==2020)$value)
7 f24 <- mean(filter(cpi, year(date)==2024)$value)
8 rate <- f24/f20
9
10 cost25 <- list(CO2=130, CH4=1590, N2O=39972)*rate
11 cost50 <- list(CO2=205, CH4=3547, N2O=65635)*rate
12
13 interp.cost <- function(y, start=2025, end=2050, c0, c1) {
14   c0 + (c1-c0)*(y-start)/(end-start)
15 }
16 GHG_Costs <- data.table(Year=2025:2050) [
17   , `:=`(
18     CO2.cost = interp.cost(Year, c0=cost25$CO2, c1=cost50$CO2),
19     CH4.cost = interp.cost(Year, c0=cost25$CH4, c1=cost50$CH4),
20     N2O.cost = interp.cost(Year, c0=cost25$N2O, c1=cost50$N2O)
21   )]

```

NPV calculation:

```

1 # after merging annual_total_CO2, total_CH4_eq, total_N2O_eq and GHG_Costs:
2 dt[, annual_cost := total_CO2*CO2.cost + total_CH4_eq*CH4.cost + total_N2O_eq*N2O.cost]
3 calc_npv <- function(d) sum(d$annual_cost/(1+discount_rate)^(d$Year-base_year))
4
5 # per simulation & pathway
6 npv_list <- dt[, .(NPV=calc_npv(.SD)), by=.(Simulation, Pathway)]
7
8 fwrite(npv_list[, .(
9   NPV_min = min(NPV),
10  NPV_mean = mean(NPV),
11  NPV_max  = max(NPV)
12 ), by=Pathway],
13   "GHG_Emissions.csv")
14
15 fwrite(npv_list, "GHG_Emissions_per_simulation.csv")

```

7.5 Air Pollutant Emissions Costs

7.5.1 Overview

The `10_Air_emissions.R` script calculates costs associated with air pollutant emissions. Key functionalities include

- Converting emission quantities to metric tons for consistency.
- Adjusting costs to 2024 dollars using Consumer Price Index (CPI) data from the `fredr` API.
- Estimation of the net present value (NPV) of the air pollutant costs using a discount rate of 2%.

7.5.2 Detailed Explanation

1. **Emission unit conversions**:

- Emission data are converted from pounds (lbs) to US tons, and subsequently to metric tons for compatibility with international standards.

2. **CPI Adjustments**:

Costs are adjusted to 2024 dollars by calculating a conversion rate between CPI values from 2000 and 2024.

3. ****NPV Calculation****: The script calculates the NPV of air pollutant costs, enabling long-term economic analysis.

7.5.3 Key Code

Unit Conversions:

Listing 37: Converting Emission Units to Metric Tons

```
1 lbs_tons_conversion <- 1 / 2204.62 # lbs to tons
2 ton_conversion <- 0.907185 # US tons to metric tons
3
4 # Example conversion
5 emission_data[, Metric.Tons := Emissions.Lbs * lbs_tons_conversion * ton_conversion]
```

CPI Adjustments:

Listing 38: Adjusting Costs Using CPI Data

```
1 fredr_set_key("63522eae4ec927d6f1d9d86bf7826cc8")
2 cpi_data <- fredr(series_id = "CPIAUCSL", observation_start = as.Date("2000-01-01"), ...
  observation_end = as.Date("2024-01-01"))
3
4 # Calculate conversion rate
5 cpi_2000 <- filter(cpi_data, year(date) == 2000) %>% summarise(YearlyAvg = mean(value))
6 cpi_2024 <- filter(cpi_data, year(date) == 2024) %>% summarise(YearlyAvg = mean(value))
7 conversion_rate <- cpi_2024$YearlyAvg / cpi_2000$YearlyAvg
8
9 # Adjust costs
10 emission_data[, Adjusted.Cost := Original.Cost * conversion_rate]
```

NPV Calculation:

Listing 39: Calculating NPV for Air Pollutant Costs

```
1 calculate_npv <- function(dt, rate, base_year, col) {
2   npv <- sum(dt[[col]] / (1 + rate)^(dt[["Year"]] - base_year))
3   return(npv)
4 }
5
6 discount_rate <- 0.02
7 base_year <- 2024
8
9 # Example NPV calculation
10 npv_air_emissions <- calculate_npv(emission_data, discount_rate, base_year, "Adjusted.Cost")
```

7.5.4 Insights and Applications

This script supports air pollutant cost modeling by:

- Ensuring consistency in emission quantities via unit conversions.
- Adjusting historical costs to 2024 values for accurate comparisons.
- Providing a long-term economic perspective through NPV calculations.

These analyses are essential for assessing the economic impact of air pollutant emissions and supporting policy-making in decarbonization strategies.

7.6 Unmet Demand Penalty Costs

7.6.1 Overview

The `11_Unmet_Demand.R` script now:

- Loads annual shortage results from a CSV (`Yearly_Results_Shortages.csv`).

- Converts unmet-demand (MWh) into dollar penalties using an EVOLL of \$3500/MWh (2024 USD).
- Computes NPVs of those penalties, discounting at 2.5 % from a 2024 base year, for each simulation and pathway.
- Exports per-simulation NPVs and pathway-level summaries to CSV.

7.6.2 Detailed Explanation

1. **Load shortage data:** Read `Yearly_Results.Shortages.csv` into a `data.table`.
2. **Compute dollar penalties:**

$$\text{Unmet_Demand_USD_total} = \text{Unmet_Demand_total_MWh} \times 3500$$

(3500 \$/MWh in 2024 USD).

3. **NPV calculation:** Define

$$\text{NPV} = \sum_t \frac{\text{Unmet_Demand_USD_total}_t}{(1 + 0.025)^{t-2024}}$$

and compute it for each (`Simulation`, `Pathway`).

4. **Export:** Write two CSVs:

- `Unmet_demand.csv`: per-simulation NPVs (excludes zeros).
- (Optionally) summary statistics by pathway.

7.6.3 Key Code

Listing 40: Unmet-Demand Penalty NPV

```

1 library(data.table)
2 # NPV helper
3 calculate_npv <- function(dt, rate, base_year, col) {
4   sum(dt[[col]] / (1+rate)^(dt$Year-base_year))
5 }
6
7 discount_rate <- 0.025
8 base_year <- 2024
9
10 # EVOLL in $/MWh (2024 USD)
11 EVOLL_2025 <- 3500
12
13 # Load shortage results
14 file_path <- "Yearly_Results.Shortages.csv"
15 output_path <- " /Total_Costs_Results"
16 Yearly_Results <- fread(file_path)
17
18 # Compute dollar penalties
19 Yearly_Results[, Unmet_Demand_USD_total := Unmet_Demand_total_MWh * EVOLL_2025]
20
21 # Per sim and pathway NPV
22 npv_list <- list()
23 for(sim in unique(Yearly_Results$Simulation)) {
24   for(path in unique(Yearly_Results$Pathway)) {
25     dt <- Yearly_Results[Simulation==sim & Pathway==path]
26     npv <- calculate_npv(dt, discount_rate, base_year, "Unmet_Demand_USD_total")
27     if(npv!=0) {
28       npv_list[[paste(sim,path)]] <- data.table(Simulation=sim, Pathway=path, NPV=npv)
29     }
30   }
31 }

```

```

32 combined_npvs <- rbindlist(npv.list)
33
34 # Summary by pathway (in billions)
35 combined_npvs[, .(
36   mean_NPV = mean(NPV)/1e9,
37   sd_NPV   = sd(NPV)/1e9
38 ), by=Pathway]
39
40 # Save results
41 fwrite(combined_npvs, file.path(output_path, "Unmet_demand.csv"))

```

7.7 Beyond the Fence Line Costs

7.7.1 Overview

The scripts `12_Other_S3_CAN_Hydro_CostAssumptions.R` and `12_Other_S3_CAN_Hydro.R` now:

- Read decarbonization pathways to compute Quebec import capacity differences (B3vsB1).
- Derive annual hydropower build (ΔMW) from the cumulative import-capacity difference.
- Fetch CPI (2016–2024) to convert ISO-NE CAPEX (\$5537/kW) and ATB cost ranges into 2024 USD.
- Compute annual CAPEX, fixed OM (FOM), and variable OM (VOM) costs.
- Estimate CH emissions from imports (Delwiche et al. EF), project CH social costs (2025–2050), and compute NPVs.

7.7.2 Hydropower Capacity Build

- QC capacity difference: $\Delta_{\text{QC}}(t) = \text{cumsum}[\text{QC}_{\text{B3}}(t) - \text{QC}_{\text{B1}}(t)]$.
- Base hydro capacity: $H(t) = \Delta_{\text{QC}}(t)/\text{CF}$.
- Year-on-year build: $\text{Build}(t) = \max\{0, H(t) - H(t-1)\}$.

7.7.3 Cost Calculations

- CPI rate: $r = \text{CPI}_{2024}/\text{CPI}_{2016}$.
- Unit CAPEX: $\text{CAPEX}_{\text{unit}} = \$5537/\text{kW} \times 1000 \times r$.
- FOM rate: $\{1.5\%, 2.5\%\} \times \text{CAPEX}_{\text{unit}}$.
- VOM: \$0.58/MWh.
- Annual costs:

$$\begin{aligned}
 C_{\text{CAPEX}}(t) &= \text{Build}(t) \times \text{CAPEX}_{\text{unit}}, \\
 C_{\text{FOM}}(t) &= \text{QC}_{\text{cap_MW}}(t) \times \text{FOM}_{\text{rate}}/\text{CF}, \\
 C_{\text{VOM}}(t) &= \text{QC}_{\text{MWh}}(t) \times \text{VOM}/\text{CF}.
 \end{aligned}$$

7.7.4 CH₄ Emissions and NPV

- CH (tonnes): $\text{Imports}_{\text{QC_TWh}} \times \text{EF} \times \frac{16}{12} \times 10^{-3}$.
- Project CH cost curve (2025–2050) via CPI-adjusted endpoints and linear interpolation.
- NPV at 2.5 %:

$$\text{NPV} = \sum_{t=2025}^{2050} \frac{C_t}{(1 + 0.025)^{t-2024}}.$$

- Export CAPEX/FOM/VOM and CH NPVs by pathway to CSV.

7.8 Total Costs

7.8.1 Overview

The `All_Costs_Tabulated.R` script automates assembling every cost element into two final tables:

- `All_Costs_per_Simulation.csv`: per-simulation, per-pathway NPVs for CAPEX, FOM, VOM, fuel, imports, GHG, air emissions, unmet demand and Canada-side costs, plus total and B1-difference.
- `All_Costs.csv`: across-simulation means by pathway and cost type (CAPEX, Fixed O&M, etc.), after dropping outlier simulations.

7.8.2 Detailed Explanation

1. ****Load component CSVs**** List all “*.csv” in the results folder, read each with `fread()`, assign by `basename`.
2. ****Summarize each cost category****
 - CAPEX&FOM: bind fossil, non-fossil, imports, group by `Simulation+Pathway+scenario`, sum NPVs.
 - VOM, Fuel, Imports, GHG, Air, Unmet: similar group-summaries.
 - Canada side: merge hydropower CAPEX/FOM/VOM and CH NPVs with adjusted QC imports.
3. ****Merge all**** Full-join per-category tables on (`Simulation`, `Pathway`).
4. ****Split B3**** Replace “B3” with “B3(1)” (zero Canada costs) and “B3(2)” (zero imports).
5. ****Compute totals**** Sum all “_mean_bUSD”/“_max_bUSD”/“_min_bUSD” columns to get per-pathway NPV and B1-differences.
6. ****Filter outliers aggregate**** Pivot long, compute per-sim total cost, drop sims outside $1.5 \times \text{IQR}$, then average by pathway+cost type.
7. ****Save CSVs**** Write both detailed and aggregated tables to disk.

7.8.3 Key Code

Listing 41: Core consolidation steps

```

1 # 1. Read all cost CSVs
2 csvs <- list.files(folder_path, "\\*.csv$", full.names=TRUE)
3 tables <- lapply(csvs, fread)
4 names(tables) <- tools::file_path_sans_ext(basename(csvs))
5
6 # 2. Summarize CAPEX & FOM
7 CAPEX_FOM <- bind_rows(tables$CAPEX.Fixed.Fossil,
8                       tables$CAPEX.Fixed.Non.Fossil,
9                       tables$CAPEX.FOM.Imports) %>%
10   group_by(Simulation, Pathway) %>%
11   summarize(across(c(NPV_CAPEX, NPV_FOM), sum), .groups="drop")
12
13 # 3. Summarize other costs similarly...
14 #   (VOM, Fuel, Imports, GHG, Air, Unmet, CAN)
15
16 # 4. Merge everything
17 All_Costs <- reduce(list(CAPEX_FOM, total_VOM, total_Fuel,
18                         total_Imports, total_GHG,
19                         total_Air, total_Unmet, total_CAN),
20                    full_join, by=c("Simulation", "Pathway"))
21
22 # 5. Split B3      B3(1)/B3(2), compute totals and B1 differences
23 All_Costs_final <- All_Costs %>%

```

```

24 # ...split logic...
25 rowwise() %>%
26 mutate(
27   Total_mean = sum(c_across(ends_with("_mean_bUSD")), na.rm=TRUE),
28   B1_diff_mean = Total_mean - Total_mean[Pathway=="B1"]
29 ) %>%
30 ungroup()
31
32 # 6. Drop outlier sims and average by pathway+cost type
33 long <- All_Costs_final %>%
34   pivot_longer(cols=ends_with("_bUSD"),
35                 names_to=c("Type", "Stat"), names_pattern="(.*)-(mean|min|max)_bUSD",
36                 values_to="Value")
37 # identify outliers by total, then
38 combined_ranges <- long %>%
39   filter(!Simulation %in% outlier_sims) %>%
40   group_by(Pathway, Type, Stat) %>%
41   summarize(Mean_Value=mean(Value), .groups="drop")
42
43 # 7. Write results
44 fwrite(All_Costs_final, file.path(output_path, "All_Costs_per_Simulation.csv"))
45 fwrite(combined_ranges, file.path(output_path, "All_Costs.csv"))

```

7.8.4 Insights and Applications

This pipeline:

- Ensures every cost driver is on the same footing and traceable back to its source CSV.
- Produces both granular (per-simulation) and summary (mean by pathway) cost tables.
- Identifies and removes simulation outliers for robust scenario comparison.
- Outputs ready-to-plot data for stacked-bar and error-bar visualizations in `ggplot2`.

7.9 CPI and Other Cost Variables

7.9.1 Overview

The `CPI_Electricity.R` script retrieves Consumer Price Index (CPI) data from the Bureau of Labor Statistics (BLS) API. It focuses on CPI series related to electricity costs across multiple regions. Key functionalities include:

- Retrieving regional CPI data for electricity using the BLS API.
- Structuring the retrieved data into a tabular format.
- Storing the processed data for integration into cost modeling workflows.

7.9.2 Detailed Explanation

1. ****API Integration****: The script uses the `blsAPI` and `rjson` libraries to query the BLS API for CPI series data.
2. ****Regions and Series****: The script specifies electricity-related series IDs and their corresponding regions, such as New England, Boston, and Middle Atlantic.
3. ****Data Processing****: Retrieved data is organized into a data frame with fields for year, period, and CPI values.

7.9.3 Key Code

API Query and Data Retrieval:

Listing 42: Retrieving CPI Data Using BLS API

```

1 library(blsAPI)
2 library(rjson)
3
4 api_key <- "a2c1e702f81947118e83cb8d029d6623"
5 series_ids <- c("APU011072610", "APUS11A72610", "APU012072610", "APUS12A72610", ...
6               "APUS12B72610")
7 regions <- c("New England", "Boston", "Middle Atlantic", "New York", "Philadelphia")
8
9 payload <- list(
10   'seriesid' = series_ids,
11   'regions' = regions,
12   'startyear' = '2022',
13   'endyear' = '2024',
14   'registrationkey' = api_key
15 )
16 response <- blsAPI(payload)
17 json_data <- fromJSON(response)

```

Data Organization:

Listing 43: Organizing CPI Data into a Data Frame

```

1 cpi_data <- do.call(rbind, lapply(json_data$Results$series, function(series) {
2   data.frame(
3     seriesID = series$seriesID,
4     Region = series$regions,
5     year = sapply(series$data, function(x) x$year),
6     period = sapply(series$data, function(x) x$period),
7     value = sapply(series$data, function(x) as.numeric(x$value))
8   )
9 })))

```

7.10 Supplemental Costs Calculation

7.10.1 Overview

The `Supplemental_Costs_calc.R` script calculates supplemental costs using GDP, population, and household data. Key functionalities include:

- Loading and filtering population and GDP growth data for the years 2025–2050.
- Converting GDP and household data to 2023 dollars using CPI adjustments.
- Preparing the processed data for integration into demand and cost models.

7.10.2 Detailed Explanation

1. ****Data Loading****: Population and GDP data are loaded from an Excel file and filtered for the target years (2025–2050).
2. ****CPI Adjustments****: CPI data is retrieved from the FRED API and used to adjust GDP values from 2012 to 2023 dollars.
3. ****Data Preparation****: The processed data is stored for further analysis in supplemental cost modeling.

7.10.3 Key Code

Loading and Filtering Data:

Listing 44: Loading Population and GDP Data

```

1 path <- "/path/to/population-GDP-growth.xlsx"
2
3 household_data <- read_excel(path)
4 household_data <- household_data %>%
5   filter(Year ≥ 2025 & Year ≤ 2050)
6 colnames(household_data) <- c("Year", "n.households.millions", "GDP_bUSD_2012")

```

CPI Adjustments:

Listing 45: Adjusting GDP to 2023 Dollars

```

1 library(fredr)
2
3 fredr_set_key("63522eae4ec927d6f1d9d86bf7826cc8")
4 cpi_data <- fredr(series_id = "CPIAUCSL", observation_start = as.Date("2000-01-01"), ...
5   observation_end = as.Date("2024-01-01"))
6
7 cpi_2012 <- filter(cpi_data, year(date) == 2012) %>% summarise(YearlyAvg = mean(value))
8 cpi_2023 <- filter(cpi_data, year(date) == 2023) %>% summarise(YearlyAvg = mean(value))
9
10 conversion_rate <- cpi_2023$YearlyAvg / cpi_2012$YearlyAvg
11 household_data$GDP_bUSD_2023 <- household_data$GDP_bUSD_2012 * conversion_rate

```

7.10.4 Insights and Applications

These scripts provide critical data processing capabilities:

- **CPI.Electricity.R:** Supports cost modeling by providing up-to-date CPI data for electricity across key regions.
- **Supplemental.Costs.calc.R:** Prepares supplemental cost data, including GDP and household data, adjusted for inflation to 2023 values.

The outputs are essential for integrating economic and demographic variables into energy modeling frameworks.

8 Ecological Impacts

This section documents four Python scripts that quantify ecological impacts—land-use change, avian mortality, water withdrawals, and population exposure—under each decarbonization pathway. Each script follows a Monte Carlo framework to capture uncertainty and produces both a stacked-bar figure with 90

8.1 Land-use Change

Overview `land_use.py` estimates additional habitat loss (million ha) from new capacity (2025→2050) in Solar, Onshore Wind, Canadian Hydro, SMRs and new gas.

Steps

1. Load all pathway sheets and stack into one DataFrame.
2. Inject “Canadian Hydro” additions for B3 manually (0 MW→3692 MW).
3. Compute capacity per technology and pathway.
4. For $n = 1000$ sims, sample land-use factors (fixed, uniform or gamma) per MW, multiply by MW, convert to million ha.
5. Compute per-tech means and total 5/95
6. Plot stacked-bar + asymmetric error bars; save `land_use.png`.
7. Write `land_use_summary.xlsx`.

Key Code

Listing 46: Monte Carlo sampling & bar plot

```

1  # compute capacity
2  df0 = decarb.query("Year==2025").set_index('Pathway')
3  df1 = decarb.query("Year==2050").set_index('Pathway')
4  cap_diff = df1[techs] - df0[techs]
5
6  # Monte Carlo
7  samples = np.zeros((n_sims, len(pathways), len(techs)))
8  for s in range(n_sims):
9      # sample factors
10     samp = np.random.gamma(alpha, theta, size=len(cap_diff))
11     land_diff = cap_diff.values * samp[:, None]
12     samples[s, :, :] = land_diff / 1e6
13
14 # means & CI
15 mean_df = pd.DataFrame(samples.mean(0), index=pathways, columns=techs)
16 low, high = np.percentile(samples.sum(2), [5, 95], axis=0)
17
18 # plot
19 mean_df.plot(kind='bar', stacked=True)
20 plt.errorbar(x, mean.sum, yerr=[mean.sum-low, high-mean.sum], fmt='none')
21 plt.savefig('land.use.png')

```

8.2 Avian Mortality

Overview `avian_mortality.py` estimates millions of bird and bat deaths from Onshore Wind and Solar capacity additions.

Steps

1. Stack pathway data and inject B3 Hydro as above.
2. Compute MW for Onshore Wind (“bird” and “bat”) and Solar (“bird”).
3. For $n = 1000$, sample mortality rates from Gamma/lognormal distributions.
4. Multiply rates by MW, sum per pathway, convert to millions.
5. Compute mean and 5/95
6. Tabulate per-category and total 10/90

Key Code

Listing 47: Sampling bird/bat mortality

```

1  for s in range(n_sims):
2      bird_on = np.random.gamma(0.20, 2.54)
3      bat_on  = np.random.gamma(1.538, 4.160)
4      s_rate  = np.random.lognormal(np.log(1.214), 1.409)
5      samples[s, :, 0] = bird_on * cap_diff["Onshore Wind"]
6      samples[s, :, 1] = bat_on  * cap_diff["Onshore Wind"]
7      samples[s, :, 2] = s_rate  * cap_diff["Solar"]
8  # plotting as above...

```

8.3 Water Withdrawals

Overview `water_use.py` computes water withdrawals (trillion gal) for new SMRs and gas turbines (2025–2050).

Steps

1. Read yearly dispatch results (2025–2050), sum SMRTWh and GasTWh per sim–pathway.
2. Multiply by water-use factors (gallons/MWh), convert to trillion gal.
3. Compute per-pathway mean and 10/90

4. Summarize in `water_withdrawals_summary.xlsx`.

Key Code

Listing 48: Water withdrawals calculation

```
1 sim_path = yr.groupby(['Simulation', 'Pathway'])[['SMR.TWh', 'New.Fossil.Fuel.TWh']].sum()
2 water = sim_path.mul(pd.Series({'SMR.TWh':740, 'New.Fossil.Fuel.TWh':32}))
3 water_tr = water/1e6 # trillion gallons
4 mean_tech = water_tr.groupby('Pathway').mean()
5 low = water_tr.groupby('Pathway').quantile(0.10)
6 high = water_tr.groupby('Pathway').quantile(0.90)
7 # plot as before...
```

8.4 Population Exposure (Viewshed)

Overview `viewshed_exposure.py` estimates how many people (million) live within project buffer zones for new imports, offshore wind and solar.

Steps

1. Compute MW for each of: Imports QC, Imports NYISO, Offshore Wind, Solar.
2. Load New England and source-region polygons; open LandScan raster.
3. For $n = 10$ sims and each pathway×tech: sprinkle one point per 1 GW, buffer by tech-specific radius, mask raster, sum population.
4. Convert to million people; compute mean and 10/90
5. Plot stacked-bar + CI; save `population_exposure.png`.
6. Save Excel summary `viewshed_exposure_summary.xlsx`.

Key Code

Listing 49: Monte Carlo exposure via raster masking

```
1 for s in range(n_mc):
2     for i, pw in enumerate(pathways):
3         for j, tech in enumerate(techs):
4             Δ = cap_diff_all.at[pw, tech]
5             n_proj = int(np.ceil(Δ/proj_size[tech]))
6             # random points in region...
7             for p in pts:
8                 buf = p.buffer(bufers_m[tech])
9                 arr, _ = rasterio.mask.mask(src, [buf], crop=True, nodata=0)
10                pop_sum += arr.sum()
11                mc[s, i, j] = pop_sum/1e6
```

8.4.1 Insights

- **Land-use:** Solar and onshore wind drive the largest habitat conversion, with B3 highest overall.
- **Avian mortality:** Onshore wind causes several million bird and bat fatalities; Solar adds less but non-negligible.
- **Water withdrawals:** SMR cooling dominates water use vs. dry-cooled gas.
- **Population exposure:** Imports and offshore wind buffers potentially affect millions of residents in New England.

This is some text.

Usage Rights and Copyright Notice: Copyright © 2025 Gazar et al. All rights reserved. This work is licensed under <http://creativecommons.org/licenses/by/4.0/> Creative Commons Attribution 4.0 International License.

